

Assignment-1.5

Name: kasuri Rohith

Hall ticket : 2303a510i2

Batch : 23

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

❖ Scenario

You are developing a basic text-processing utility for a messaging application.

❖ TaskDescription

Use GitHub Copilot to generate a Python program that:

- Reverses a given string
 - Accepts user input
 - Implements the logic directly in the main code
 - Does not use any user-defined functions
- #### ❖ Expected Output
- Correct reversed string
 - Screenshots showing Copilot-generated code suggestions
 - Sample inputs and outputs

Prompt : write a python program a string reversal without function

Code:

```
1  text = input("Enter a string: ")
2  reversed_text = ""
3  for i in range(len(text) - 1, -1, -1):
4      reversed_text += text[i]
5  print(reversed_text)
6
```

Output

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/assign.py
Enter a string: hello
olleh
```

Task 2: Efficiency C Logic Optimization (Readability Improvement)

❖ Scenario

The code will be reviewed by other developers.

❖ TaskDescription

Examine the Copilot-generated code from Task 1 and improve it by:

- Removing unnecessary variables
- Simplifying loop or indexing logic
- Improving readability
- Use Copilot prompts like:
 - “Simplify this string reversal code”
 - “Improve readability and efficiency”

Hint:

Prompt Copilot with phrases like

“optimize this code”, “simplify logic”, or “make it more readable”

❖ Expected Output

- Original and optimized code versions
- Explanation of how the improvements reduce time complexity

Prompt: In the above code string reversal without function add optimized string reversal (readability C efficiency)

Code:

```
#Task 2: Optimized string reversal (readability & efficiency)

text = input("Enter a string: ")
print("Reversed string:", text[::-1])
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/assign.py
Enter a string: python
nohtyp
```

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

❖ Scenario

The string reversal logic is needed in multiple parts of an application.

❖ TaskDescription

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to reverse a string
- Returns the reversed string
- Includes meaningful comments (AI-assisted)

❖ Expected Output

- Correct function-based implementation
- Screenshots documenting Copilot's function generation
- Sample test cases and outputs

Prompt: in the above it should include modular design using functions

Code:

```
# Task 3: Modular design using functions

def reverse_string(text):
    """
    Reverse the given string.

    Args:
        text (str): The string to reverse.

    Returns:
        str: The reversed string.
    """
    return text[::-1]

user_input = input("\nEnter a string for Task 3: ")
print("Task 3 - Reversed string:", reverse_string(user_input))
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/assign.py
Enter a string for Task 3: mango
Task 3 - Reversed string: ognam
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 4: Comparative Analysis - Procedural vs Modular Approach (With vs Without Functions)

❖ Scenario

You are asked to justify design choices during a code review.

❖ TaskDescription

Compare the Copilot-generated programs:

➤ Without functions (Task 1)

➤ With functions (Task 3)

Analyze them based on:

➤ Code clarity

➤ Reusability

➤ Debugging ease

➤ Suitability for large-scale applications

❖ Expected Output

Comparison table or short analytical report

Prompt: in the above it should include comparative analysis-procedural approach

Code:

```

29 # Task 4: Comparative Analysis - Procedural vs Modular Approach
30
31 # --- Procedural Approach (Without Functions) ---
32
33 text = input("Enter a string for procedural approach: ")
34 reversed_text = ""
35
36 for i in range(len(text) - 1, -1, -1):
37     reversed_text += text[i]
38
39 print("Procedural result:", reversed_text)
40
41
42 # --- Modular Approach (With Functions) ---
43
44 def reverse_string(text):
45     """
46     Reverse the given string.
47     Args:
48     |   text (str): The string to reverse.
49     Returns:
50     |   str: The reversed string.
51     """
52     return text[::-1]
53
54 user_input = input("\nEnter a string for modular approach: ")
55 print("Modular result:", reverse_string(user_input))

```

Output:

```

Procedural result: watch:tluser larudecorPhctaw

Enter a string for modular approach: watch
Modular result: hctaw
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal)

❖ Scenario

Your mentor wants to evaluate how AI handles alternative logic paths.

❖ TaskDescription

Prompt GitHub Copilot to generate:

- A loop-based string reversal approach
- A built-in / slicing-based string reversal approach

❖ Expected Output

- Two correct implementations
- Comparison discussing:
 - Execution flow
 - Time complexity
 - Performance for large inputs
 - When each approach is appropriate

Prompt: in the above it should include itractive vs slicing approaches

Code:

```
57 # Task 5: Iterative vs Slicing approaches
58
59 # Iterative (loop-based)
60 text = input("\nEnter a string for Task 5 (Iterative): ")
61 reversed_text = ""
62
63 for char in text:
64     reversed_text = char + reversed_text
65
66 print("Task 5 (Iterative) - Reversed string:", reversed_text)
67
68
69 # Slicing (built-in)
70 text = input("\nEnter a string for Task 5 (Slicing): ")
71 print("Task 5 (Slicing) - Reversed string:", text[::-1])
72
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/assign.py

Enter a string for Task 5 (Iterative): bat
Task 5 (Iterative) - Reversed string: tab

Enter a string for Task 5 (Slicing): cat
Task 5 (Slicing) - Reversed string: tac
```